

Fire Fighter System Monitor

Laporan dokumentasi proyek lengkap — IoT · ESP32 · Supabase · Next.js · Vercel

Penyusunan: 2026

Daftar isi

1. [Pendahuluan](#)
2. [Rumusan masalah dan tujuan](#)
3. [Tinjauan komponen hardware & software](#)
4. [Arsitektur sistem](#)
5. [Alur data \(end-to-end\)](#)
6. [Cara kode berjalan — firmware ESP32](#)
7. [Diagram alur loop ESP32](#)
8. [Cara kode berjalan — dashboard Next.js](#)
9. [Alur pengguna membuka dashboard](#)
10. [Basis data Supabase](#)
11. [Pengujian & troubleshooting](#)
12. [Kesimpulan & saran](#)
13. [Referensi](#)

1. Pendahuluan

Sistem deteksi kebakaran dini dan respons otomatis (misalnya aktifkan pompa) penting untuk mengurangi risiko kerugian. Dengan **Internet of Things (IoT)**, sensor dan aktuator di lapangan dapat dipantau dari jarak jauh. Proyek ini menghubungkan **ESP32** sebagai node lapangan, **Supabase** sebagai basis data dan saluran *realtime*, serta aplikasi web **Next.js** yang di-*deploy* di **Vercel**.

Laporan menjelaskan **komponen apa saja** yang dipakai, **alur data** dari sensor hingga browser, dan **cara kode dijalankan** pada firmware maupun frontend.

2. Rumusan masalah dan tujuan

Rumusan masalah

- Bagaimana data sensor/aktuator terlihat **realtime** di dashboard?
- Bagaimana **satu basis data** aman untuk embedded dan browser?
- Bagaimana **reset WiFi** ESP dapat diminta dari tanpa kabel UART?

Tujuan

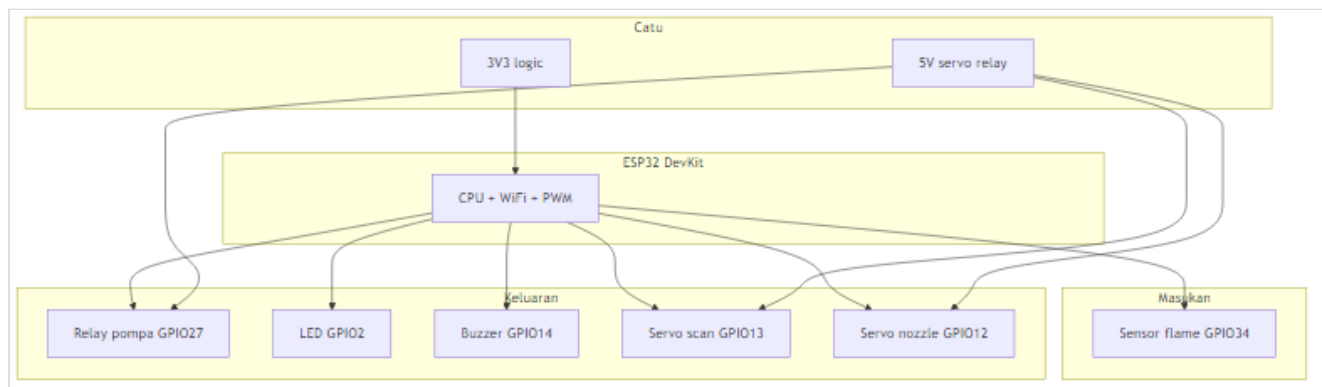
1. Merancang firmware ESP32 dengan filter sensor dan kendali servo, relay, alarm.

2. Mengintegrasikan penyimpanan & realtime dengan Supabase.
3. Membangun antarmuka Next.js yang bereaksi terhadap perubahan data.
4. Mendokumentasikan arsitektur untuk akademik / pemeliharaan.

3. Tinjauan komponen

3.1 Komponen hardware

No	Komponen	Fungsi	Catatan
1	ESP32	MCU + WiFi, jalankan firmware	Stabilkan catu & GND
2	Sensor api (pin 34)	Deteksi api (digital)	GPIO34 input-only
3	Relay (pin 27)	Pompa / beban	Active low di firmware
4	Servo ×2 (13, 12)	Scan & nozzle	Catu 5 V terpisah cukup ampere
5	LED (pin 2)	Indikator	—
6	Buzzer (pin 14)	Alarm	—
7	Catu daya	5 V servo/relay, 3V3 logika	GND bersama



Gambar 1 — Blok komponen hardware: hubungan ESP32, sensor, aktuator, dan catu.

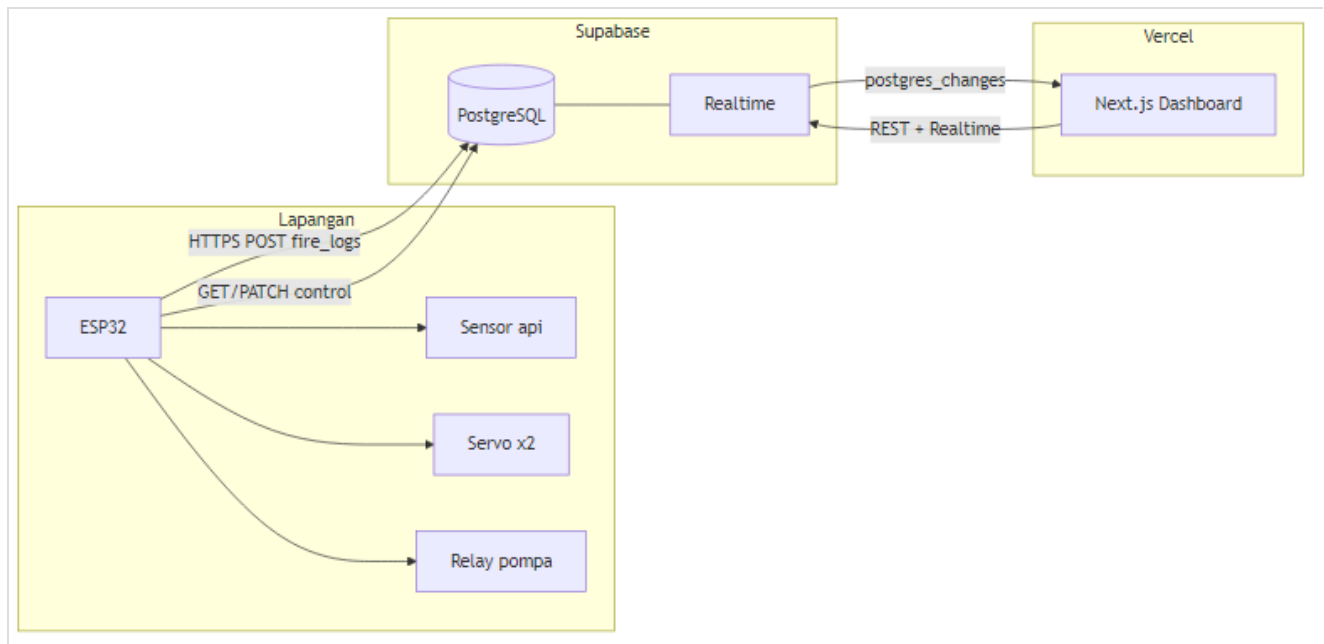
3.2 Komponen software

Komponen	Peran
Arduino Core ESP32 + WiFiManager	Firmware & portal WiFi
ESP32Servo	PWM servo
HTTPClient	REST ke Supabase
Supabase	PostgreSQL, PostgREST, Realtime
Next.js 16 / React 19	UI dashboard
Supabase JS	Query + subscription

Komponen	Peran
Vercel	Hosting web

4. Arsitektur sistem

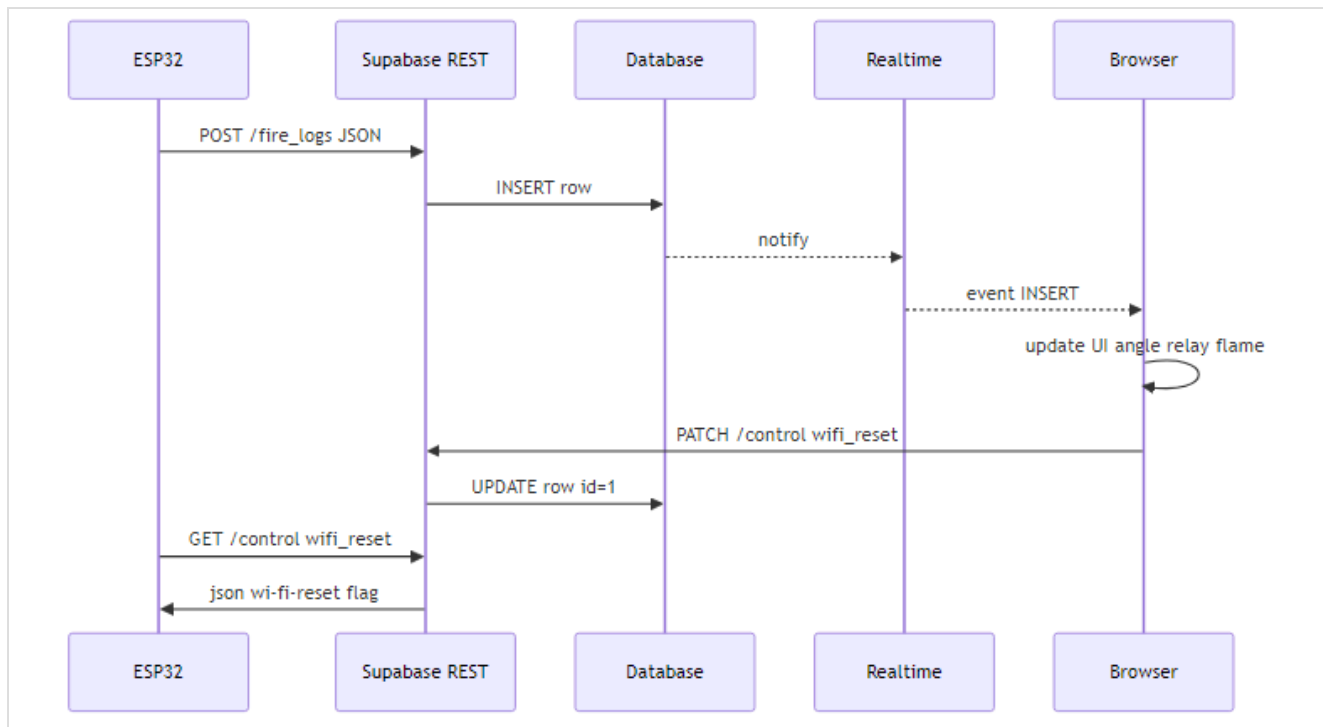
Seluruh komunikasi melewati **Supabase**. ESP tidak memanggil domain Vercel; browser dan ESP sama-sama memakai URL & kunci project Supabase.



Gambar 2 — Arsitektur: ESP32 ↔ Supabase ↔ Browser (Next.js di Vercel).

5. Alur data (end-to-end)

Urutan pesan saat log masuk dan saat kontrol/reset WiFi lewat tabel `control1` :



Gambar 3 — Diagram urutan: POST fire_logs, Realtime ke browser, GET/PATCH control.

1. ESP **POST** JSON ke `/rest/v1/fire_logs`.
2. Baris tersimpan; Realtime mendorong ke klien yang berlangganan.
3. Browser memperbarui state UI (sudut, api, relay, log).
4. Untuk reset WiFi: UI **PATCH** `control`; ESP **GET** periodik memeriksa `wifi_reset`.

6. Cara kode berjalan — firmware ESP32

Berkas: `esp32_fire_system.ino`.

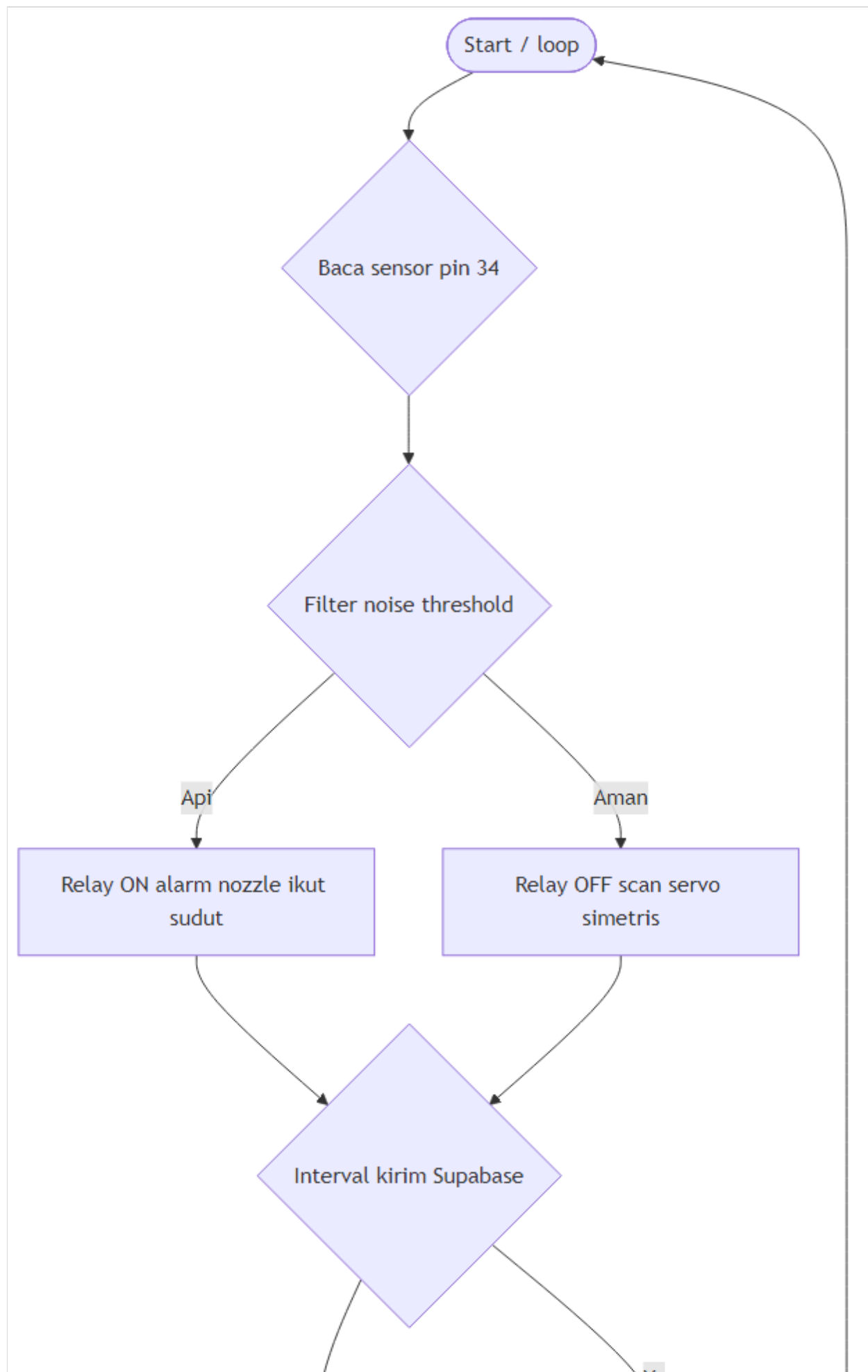
setup()

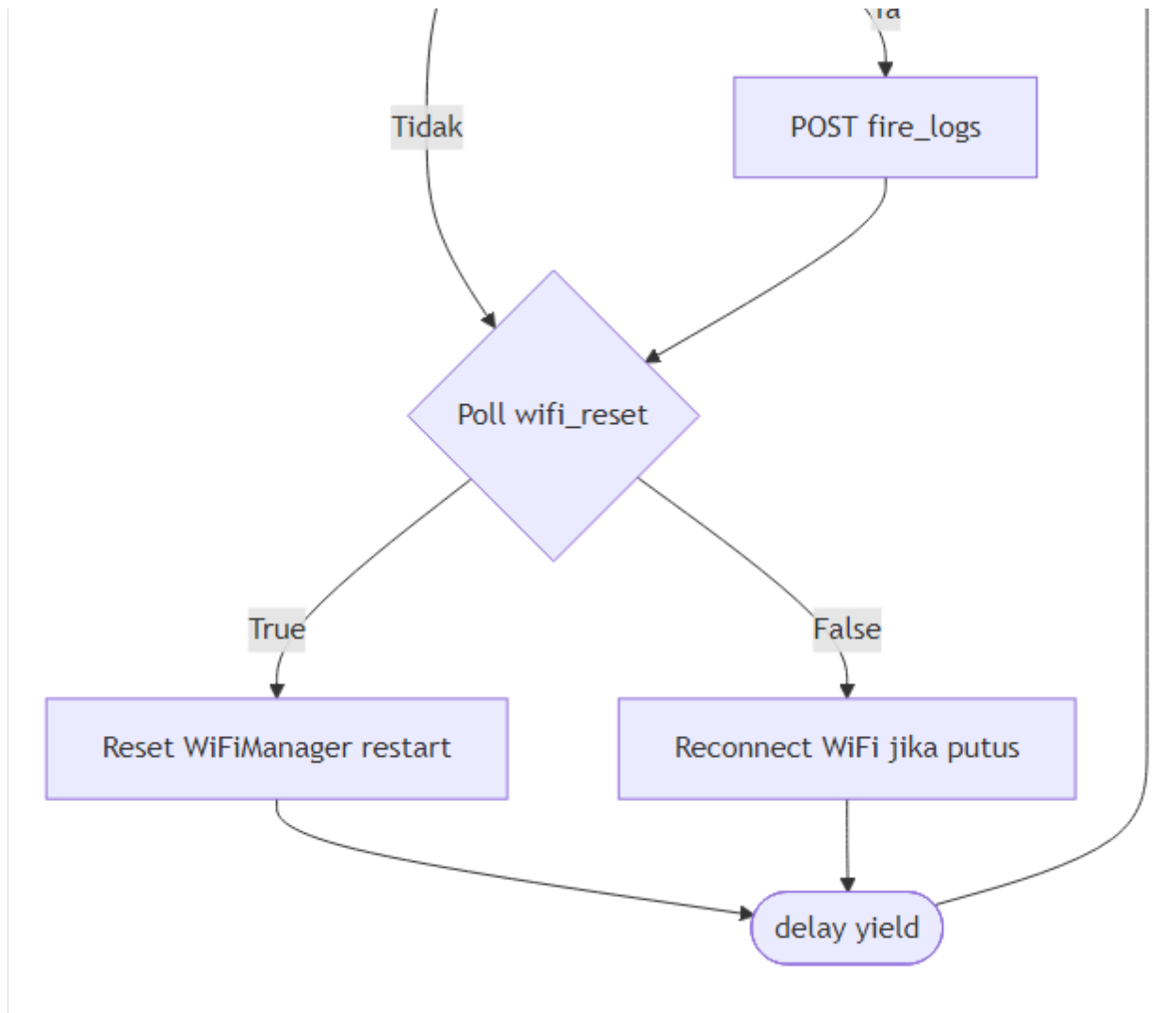
- Inisialisasi pin, servo (kalibrasi PWM), posisi awal tengah.
- WiFiManager membuka hotspot konfigurasi (mis. SSID `FIRE_SYSTEM_FIKRI`) bila perlu.

loop()

- Reconnect WiFi jika putus (periodik).
- Baca sensor + filter *threshold*.
- Mode api: relay ON, alarm terjadwal, nozzle ikut sudut; mode aman: relay OFF, servo scan simetris dengan pembatasan frekuensi tulis PWM.
- POST `fire_logs` pada interval.
- Poll `wifi_reset` pada `control`.
- `yield()` + `delay` kecil agar stack WiFi stabil.

7. Diagram alur utama loop ESP32





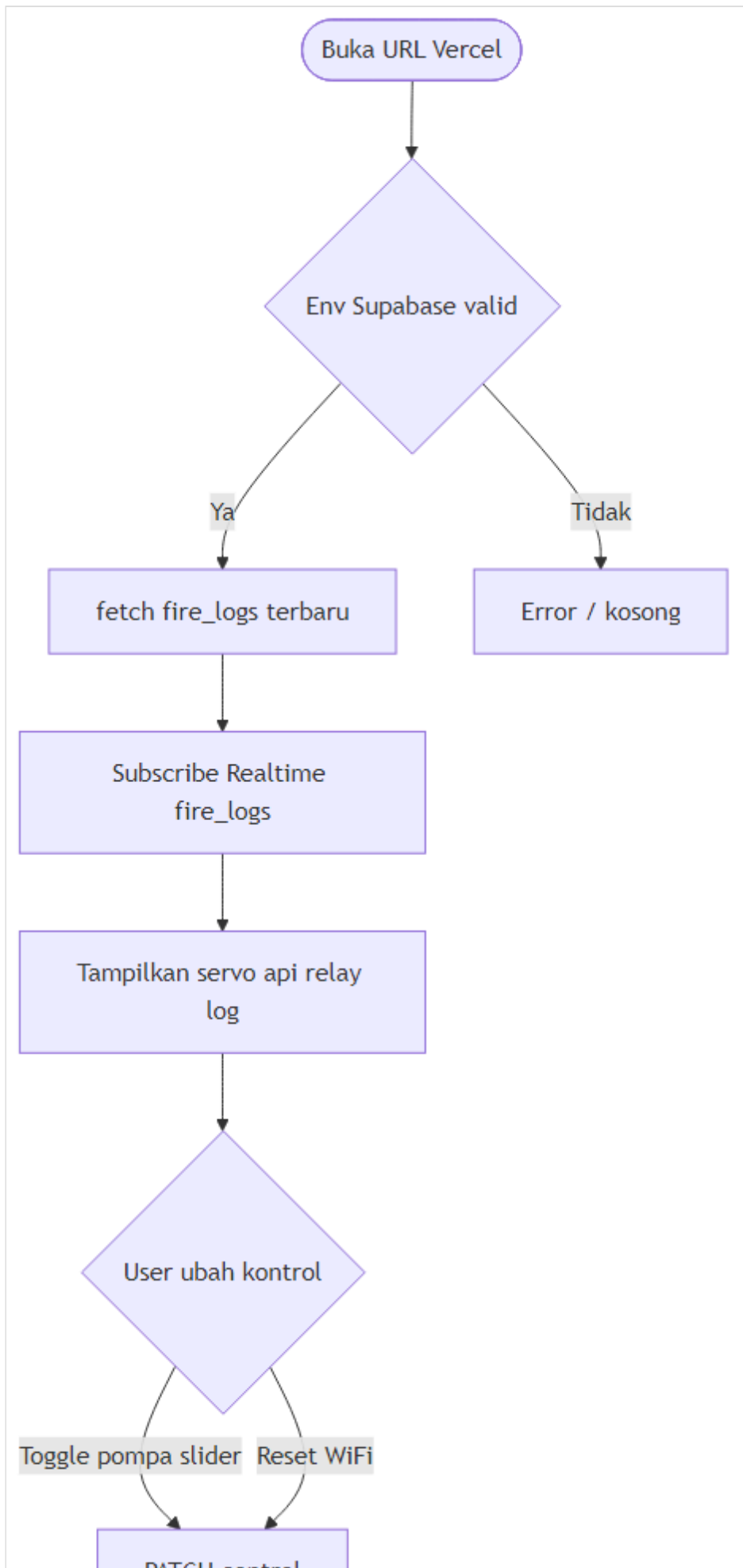
Gambar 4 — Keputusan utama dalam loop firmware (ringkasan).

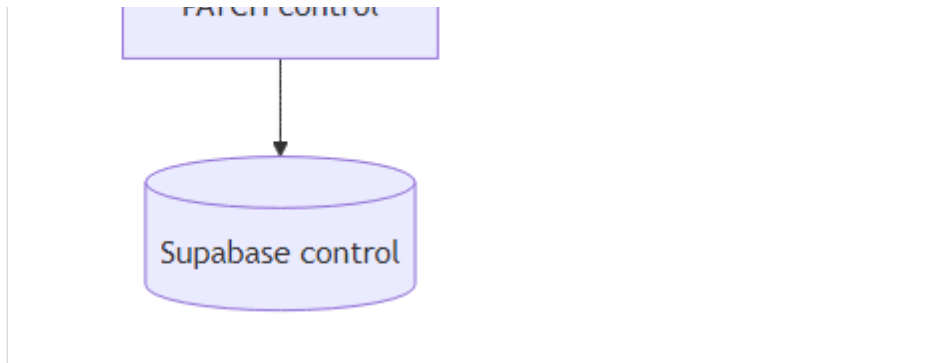
8. Cara kode berjalan — dashboard (Next.js)

Berkas utama: `app/page.tsx` (*client component*).

- Saat *mount*: fetch baris terbaru `fire_logs`, fetch log terkini, subscribe Realtime.
- Interaksi: toggle/slider memanggil `PATCH` ke `control?id=eq.1`; tombol reset mengirim `wifi_reset`.
- Status "perangkat online" memakai usia `created_at` data terakhir (bukan ping WiFi langsung).

9. Alur pengguna membuka dashboard





Gambar 5 — Alur pengguna dari membuka URL hingga aksi kontrol.

10. Basis data Supabase

- **fire_logs**: histori pengukuran (`flame` , `relay` , `angle` , `created_at` , dll.).
- **control**: biasanya satu baris (`id=1`) untuk perintah UI, termasuk `wifi_reset` .
- **RLS**: policy harus mengizinkan operasi yang dipakai kunci anon sesuai kebijakan Anda.

11. Pengujian & troubleshooting

Gejala	Kemungkinan	Tindakan
401 dari ESP	URL/key salah	Salin anon key & URL satu project
Insert ditolak	RLS	Policy INSERT/SELECT
UI tidak live	Realtime	Aktifkan replikasi tabel untuk Realtime
Servo bergemeter	Catu lemah	Catu 5 V terpisah, GND bersama

12. Kesimpulan & saran

Sistem mengintegrasikan ESP32, Supabase, dan Next.js melalui pola **pusat data tunggal**. Firmware bekerja dalam **loop waktu-nyata** dengan tugas jaringan terjadwal; aplikasi web **reaktif** terhadap peristiwa basis data.

Saran: kolom `device_id` untuk multi-board; firmware membaca `control` agar kontrol web sama dengan hardware; autentikasi untuk produksi.

13. Referensi

- Next.js — <https://nextjs.org/docs>
- Supabase — <https://supabase.com/docs>
- ESP32 Arduino, WiFiManager, dokumentasi servo PWM.